CS 3892 / CS 5892 — Special Topics

Formal Methods: Safe and Trustworthy Artificial Intelligence

Vanderbilt University · Fall 2026 · 3 credits
Tuesday / Thursday, 11:00 a.m. – 12:15 p.m. · Featheringill Hall (FGH) 306

---

1. Instructor and course staff

Instructor: Taylor T. Johnson, PhD, PE — Professor of Computer Science, Computer Engineering, and Electrical Engineering
Email: taylor.johnson@vanderbilt.edu · Web: https://www.taylortjohnson.com

Office hours: by appointment — email to schedule. Easiest immediately before or after class; my office is in ISIS.

Teaching assistant: Michaela Winfree
Email: michaela.winfree@vanderbilt.edu · TA office hours: to be announced

Course materials and announcements: **Brightspace**. Code, assignment starters, and slides: the course GitHub repository (link on Brightspace).

2. Course description

This project course studies formal methods for establishing the trustworthiness of systems, with a focus on projects for safe and trustworthy artificial intelligence. Formal methods are mathematically rigorous techniques and tools for specifying, designing, and analyzing systems so that they demonstrably meet their requirements. We study model checking, satisfiability (SAT/SMT), reachability, abstract interpretation, automated and interactive theorem proving, and neural network verification — and, throughout, how each of these is being combined with AI systems, including large language models.

The subject has two directions and we take both seriously:

- **FM4AI** — using formal methods *on* AI systems. Verifying robustness of a neural network; proving that an agent cannot exfiltrate a secret; checking an LLM's answer against a policy with a solver rather than another model.
- **AI4FM** — using AI *to do* formal methods. Autoformalizing English requirements into temporal logic; LLM-assisted proof search in Lean; generating specifications, invariants, and counterexample explanations.

Students apply these methods in a semester-long team project. The course counts as the **CS project requirement** and may be used to satisfy Vanderbilt's **Immersion** requirement.

Prerequisites (topical): elementary discrete mathematics — propositional, first-order, and higher-order logic; set theory; Boolean operations; basic graph theory — and elementary programs and algorithms, including syntax and semantics. You should be able to write, compile, run, and debug programs in more than one language, and be comfortable at a command line. Familiarity with AI and machine learning, including generative AI and LLMs, is assumed.

Recommended course prerequisites: CS 3251 and CS 3270, plus one of CS 3262, CS 4260, CS 4262, CS 4267, or ECE 4363.

Cross-listing: CS 3892 (undergraduate) and CS 5892 (graduate) are the same course, taught together, with the same lectures, schedule, and Brightspace site. Graduate students complete additional coursework — see **§7*, Graduate students (CS 5892)***, which is where the difference is spelled out.

Why this course, and why now

Generative AI has industrialized software *production* without a matching advance in software *validation*. The gap is not hypothetical. In the twelve months before this course begins, an AI-orchestrated cyber-espionage campaign ran with 80–90% of its tactical work unsupervised; a coding agent destroyed a production database, backups included, in about nine seconds; Amazon required senior sign-off on AI-assisted changes after outages its own internal briefings tied to "Gen-AI assisted changes" with "high blast radius"; and two U.S. federal judges withdrew opinions containing hallucinated citations.

Formal methods are the part of computer science that produces *guarantees* rather than test results. This course asks what those guarantees can and cannot cover when the artifact under scrutiny is a neural network or an agent — and, symmetrically, whether AI can finally make formal methods cheap enough to use routinely.

3. Learning objectives

By the end of the course you will be able to:

1. Explain what formal methods are, what they guarantee, and — equally important — what they do not, and where they sit in a software and systems engineering process.
2. Describe AI risks and failure modes — hallucination, jailbreaking, adversarial perturbation, backdoors, prompt injection, tool-use and agentic failures, IP and provenance — and analyze real incidents against a risk taxonomy.
3. Translate informal requirements into formal specifications: invariants, pre/postconditions, and temporal logic; and assess when each formalism is expressive enough and when it is too costly.
4. Model a reactive system as a transition system and verify safety and liveness properties with a model checker, reading and explaining counterexample traces.
5. Encode decision problems for SAT and SMT solvers and explain how these engines underpin much of the field, including industrial hallucination-prevention systems.
6. Apply static analysis, dynamic analysis, and abstract interpretation to programs, and explain the precision/cost trade-off a choice of abstract domain buys.
7. Write and complete machine-checked proofs in **Lean 4**, including proving an invariant of a transition system by the inductive-invariant method, and relate the automata-theoretic foundations of model checking to their formalization in **Cslib**.
8. Verify properties of neural networks, interpret verified / falsified / timeout outcomes correctly, and state honestly where the scaling limits lie.
9. Use generative AI as an instrument of verification — autoformalization, proof search, spec generation — and *evaluate its output critically*, including detecting confident, plausible, and wrong results.

10. Execute a substantial team project and communicate it in writing, in a talk, and in a demo.

4. Course format and calendar

Two 75-minute meetings per week, Tuesday and Thursday, 11:00 a.m.–12:15 p.m., in **Featheringill Hall (FGH) 306**. Sessions mix lecture, live tool demonstrations, in-class case-study discussion, and project checkpoints. **Bring a laptop.**

Key Fall 2026 dates (Vanderbilt Registrar, 2026–27 academic calendar; calendars are subject to change):

Date	Event
Wed Aug 26	First day of classes (our first meeting is Thu Aug 27)
Fri Sep 4	Last day to add/drop
Thu–Fri Oct 22–23	Fall Break — no class Thu Oct 22
Fri Oct 30	Three separate 11:59 p.m. deadlines: last day to **withdraw**; last day to **register for pass/fail**; last day to **change pass/fail back to graded**. (Medical underload requests close the same day at 4:00 p.m.)
Mon Nov 23 – Fri Nov 27	Thanksgiving holidays, classes do not meet (registrar states the range as Nov 22–29, both Sundays) — **for this course that is Tue Nov 24 and Thu Nov 26**
Thu Dec 3	**Showcase poster / video file due** — hard deadline, department pays printing (§12)
Thu Dec 10	Last day of classes
Thu Dec 10, 5–7 p.m.	**CS Immersion Showcase**, Adams Atrium, Featheringill Hall (§12)
Fri Dec 11, Sun Dec 13	Reading days
Sat Dec 12, Mon Dec 14 – Sat Dec 19	Final examination period
Mon Dec 21	Final grades due
(Mon Dec 14, 9:00 a.m.)	*Registrar's exam slot for this course — reserved, not currently used (§7)*

There is no final exam planned; the final project presentation and deliverables take its place. See §7 for the one circumstance in which that could change, and the slot it would occupy.

> **Plan around this:** the poster deadline is **Thursday December 3**, a week before the event — and the Showcase itself falls on the **last day of classes**, the same day as the second half of final project presentations. Nothing about the last week has slack in it. Build the poster in November.

The full session-by-session schedule — every topic, homework, quiz, and project deadline — is posted on Brightspace and in the course repository.

5. Topics

Unit 0 · Framing (session 1). What formal methods are and what they buy. The verification triple: model + specification → verifier → verdict. FM4AI and AI4FM.

Unit 1 · AI failure modes, risk, and governance (sessions 2–4). A taxonomy of failure: alignment, hallucination, toxicity, jailbreaking, adversarial perturbation, backdoors and trojans, data provenance, IP, human factors. Prompt injection and agentic tool-use failures. Case studies from the past year. Guardrails, red-teaming, model cards and datasheets. Risk frameworks and regulation — NIST AI RMF (and the fact that it is currently under revision), NIST AI 100-2 adversarial-ML taxonomy, the EU AI Act as amended in July 2026, ISO/IEC 24029-2, EASA's aviation AI concept paper. The central contrast of the course, introduced early: a *filter* that another model evaluates, versus a *check* that a solver decides.

Unit 2 · Logic and preliminaries (sessions 5–6). Sets and set-builder notation; predicates; propositional, first-order, and higher-order logic; satisfiability versus validity; the duality between formulas and sets of states.

Unit 3 · SAT, SMT, and decision procedures (sessions 7–8). DPLL/CDCL; theories (linear arithmetic, bit-vectors, uninterpreted functions); DPLL(T). Z3 from Python and SMT-LIB. Encoding problems as constraints. Bounded reachability. Why SMT is the engine under industrial hallucination checking.

Unit 4 · Modeling and specification (session 9). Transition systems `T = (S, S₀, →, AP, L)`; traces and reachability; invariants and *inductive* invariants; strengthening. The SMV language.

Unit 5 · Temporal logic (sessions 10, 12). LTL and CTL; safety versus liveness; ω-words and ω-regular languages; Büchi automata and the automata-theoretic view of model checking. Signal temporal logic and quantitative robustness. Temporal properties over *agent execution traces* — the specification problem that per-request content filtering cannot express.

Unit 6 · Model checking (sessions 13–14). Explicit-state, symbolic (BDD), and bounded (SAT) model checking, and when each is the right tool. Image computation, fixpoints, quantifier elimination. nuXmv in practice; counterexample-guided debugging; k-induction and IC3.

Unit 7 · Program analysis (sessions 15–17). Programs as transition systems over memory states. Bounded model checking of C with CBMC; loop unwinding and unwinding assertions. Deductive verification, Hoare logic, loop invariants, ACSL and Frama-C. Abstract interpretation: abstract domains, Galois connections, widening — and what precision is lost at a join. Static, dynamic, and concolic analysis; automated test generation.

Unit 8 · Interactive theorem proving (sessions 18–21). Lean 4: terms, types, `def`, `theorem`, tactics. Curry–Howard. Induction. Proving invariants of transition systems, and proving *parametric* statements a model checker cannot settle. Then **Cslib** — the Lean library for computer science — where the automata-theoretic foundations of Unit 5 are machine-checked: labelled transition systems, bisimulation, Hennessy–Milner logic and the Hennessy–Milner theorem, ω-languages, Büchi acceptance, the strict inclusion of deterministic in nondeterministic Büchi automata, and closure of ω-regular languages under complementation. We will also look carefully at what Cslib *does not* yet contain — there is no LTL, no LTL→Büchi translation, and no emptiness check — because the boundary between "proved" and "asserted on a slide" is precisely what this course is about.

Unit 9 · AI for formal methods (sessions 23–24). Autoformalization: English → temporal logic, English → SMV, English → ACSL, English → Lean. LLM proof search and the current state of AI theorem proving. What is genuinely solved, what is saturated benchmark theater, and the **soundness gap**: a machine-checked proof of a machine-generated formalization certifies only that the two agree. The Erdős #728 episode — where formalization in Lean is exactly what separated a real result from a retracted claim — is the argument for this entire course, delivered by current events.

Unit 10 · Neural network verification (sessions 25–26). Robustness as a formal property. Incomplete versus complete methods; interval bound propagation, CROWN-style linear relaxation, branch-and-bound. ONNX and VNN-LIB (and the 2.0 revision, which gives the format a formal semantics mechanized in Agda). Tools: α,β-CROWN, NNV, n2v, NeuralSAT, PyRAT, Marabou. VNN-COMP 2026 results as a live dataset. An honest account of the scaling limits, and of certification practice in aviation.

Unit 11 · Project presentations (sessions 27–28). Proposal talks are session 10 and the milestone is session 22.

6. Materials

No required textbook. Everything below is free, and everything is optional except where a reading is assigned in class.

Tools

Every tool has a **zero-install path**. You will not need to build anything locally to pass this course, though you are welcome to.

Tool	Use	Zero-install
Z3	SMT solving	Z3 Guide, in-browser: https://microsoft.github.io/z3guide/ · repo: https://github.com/Z3Prover/z3
nuXmv / NuSMV	Model checking	smvis, in-browser: https://smvis-378135919048.us-central1.run.app/ · nuXmv is license-gated, so we use NuSMV where needed
Lean 4	Interactive theorem proving	https://live.lean-lang.org/ · Natural Number Game: https://adam.math.hhu.de/
Cslib	CS structures in Lean	https://github.com/leanprover/cslib — note the org is `leanprover`, **not** `leanprover-community`
CBMC	Bounded model checking of C	GitHub Codespaces (course container) · https://github.com/diffblue/cbmc
Frama-C	Deductive verification, ACSL	https://frama-c.com/
α,β-CROWN / auto_LiRPA	Neural network verification	Google Colab (course notebooks)
Course container	Everything preinstalled	GitHub Codespaces via https://github.com/ttj/fmaiv

Lean version pinning — read this before installing anything. Lean releases monthly and there will be two or three releases during the semester. For the whole term we pin **Lean v4.33.0 + Mathlib v4.33.0 + Cslib v4.33.0**. Do not follow Cslib's README, which points at `main` and currently drags in a release-candidate toolchain. Install by installing VS Code first, then the **Lean 4** extension, and letting the extension run its guided setup. Note that the in-browser Lean editors ship Mathlib only — `import Cslib` will not work there, so Cslib work happens in Codespaces or locally.

Lab spine

Much of the hands-on work is built on **FMAIV** — *Formal Methods & AI-Assisted Verification*, a short course with worked examples, notebooks, and a Codespaces container:

- Repository: https://github.com/ttj/fmaiv · Slides: https://ttj.github.io/fmaiv/

FMAIV threads a single bounded counter through Z3, nuXmv, Lean, CBMC, and Cryptol, verifying the *same* property five ways. The homeworks follow that thread; the machine and the property are defined once, in `labs/running-example.md`. By the end you will have a concrete, earned sense of what "bounded" versus "unbounded," and "checked" versus "proved," actually mean.

AI risk, standards, and governance

- Microsoft, *Failure Modes in Machine Learning* — https://learn.microsoft.com/en-us/security/engineering/failure-modes-in-machine-learning
- **NIST AI RMF 1.0** (= NIST AI 100-1) — https://www.nist.gov/itl/ai-risk-management-framework · https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf *Note: NIST states the RMF is being revised under the July 2025 AI Action Plan. We read it as a live document, not a settled one.*
- **NIST AI 600-1**, Generative AI Profile — https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf
- **NIST AI 100-2 E2025**, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations* — https://csrc.nist.gov/pubs/ai/100/2/e2025/final (use the corrected April 2025 PDF together with the June 2025 errata)
- **EU AI Act**, consolidated, and Regulation (EU) 2026/1744 (the "Digital Omnibus on AI," in force 27 July 2026) — https://eur-lex.europa.eu/eli/reg/2026/1744/oj/eng *What matters for us: Article 50 transparency obligations apply from 2 August 2026; the high-risk obligations moved to December 2027 and August 2028. Any source dating high-risk from August 2026 predates the amendment.*
- **ISO/IEC 24029-2:2023**, robustness of neural networks — methodology for the use of formal methods — https://www.iso.org/standard/79804.html. Related: ISO/IEC 42001:2023 (AI management systems); ISO/IEC 24029-3 is in development.
- **EASA** AI concept paper, Proposed Issue 03 (June 2026) — the current aviation position; AI touching DAL A or DAL B failure conditions remains outside accepted means of compliance.
- **DoD AI Assurance (AIA) Toolkit** (formerly the Responsible AI Toolkit) — https://www.tradewindai.com/aia-toolkit
- **AI Incident Database** — https://incidentdatabase.ai/ · **OECD AI Incidents Monitor** — https://oecd.ai/en/incidents · **AI Hallucination Cases Database** — https://www.damiencharlotin.com/hallucinations/

Industrial formal methods for AI

- **AWS Bedrock Automated Reasoning checks** — SMT-backed policy checking of model output, GA August 2025 — https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html
- **AWS Bedrock Guardrails** — https://aws.amazon.com/bedrock/guardrails/
- Werner Vogels interviews Byron Cook, *A chat with Byron Cook on automated reasoning and trust in AI systems*, February 2026 — the single best introductory reading for the course's thesis — https://www.allthingsdistributed.com/2026/02/a-chat-with-byron-cook-on-automated-reasoning-and-trust-in-ai-systems.html
- **Cedar Analysis** — an authorization-policy verifier whose symbolic compiler is written in Lean and discharges to CVC5, with soundness and completeness proved in Lean.
- **Dogwood** — a governance language adding past-time temporal conditions (`since`, `formerly`, `once`) over an agent's recent events, lowering to Cedar, with a trace-`replay` CLI — https://github.com/dogwood-policy/dogwood. We use it as the worked example of a specification a per-request filter cannot express.
- **Adversarial Robustness Toolbox** — https://github.com/Trusted-AI/adversarial-robustness-toolbox

Verification competitions and benchmarks

- **VNN-COMP** — https://vnn-comp.github.io/ · 2026 results: https://github.com/VNN-COMP/vnncomp2026_results · **VNN-LIB 2.0** — https://www.vnnlib.org/
- **SV-COMP** — https://sv-comp.sosy-lab.org/ · **SMT-COMP** — https://smt-comp.github.io/ · **SAT competition** — https://satcompetition.github.io/
- **ARCH** (cyber-physical systems) — https://cps-vo.org/group/ARCH/benchmarks

Reference books

- Aho & Ullman, *Foundations of Computer Science* — http://infolab.stanford.edu/~ullman/focs.html
- Alur, *Principles of Cyber-Physical Systems*, MIT Press
- Clarke, Grumberg & Peled, *Model Checking*, MIT Press
- Nipkow & Klein, *Concrete Semantics* — https://concrete-semantics.org/
- *Functional Programming in Lean* — https://lean-lang.org/functional_programming_in_lean/ (gentlest Lean entry point for CS students)
- *Theorem Proving in Lean 4* — https://lean-lang.org/theorem_proving_in_lean4/
- *The Hitchhiker's Guide to Logical Verification*, 2026 edition — https://github.com/lean-forward/logical_verification_2026
- nuXmv user manual — https://es-static.fbk.eu/tools/nuxmv/downloads/nuxmv-user-manual.pdf

7. Assignments and grading

Component	Weight
Term project	**50%**
Homeworks (5)	20%
Quizzes (5, lowest dropped)	10%
Discussion reflections (9, lowest dropped)	10%
Leading one in-class case study	5%
In-class engagement and peer feedback	5%

The last three rows are the 20% for case studies, attendance, and participation, itemized. These six rows are exactly the six categories you will see in the Brightspace gradebook, so the number you see there is the number this table describes.

There is no final exam planned. The final project presentation and deliverables take its place. However — and this is the one place the grade structure can change — **the instructor reserves the right to adjust these percentages and to add assignments, including a final examination, if class participation and in-class engagement prove insufficient.** This course is built on discussion and in-class work; if that does not materialize, the assessment has to move somewhere it can be measured. Should a final exam be added, it would be held in the registrar's assigned slot for this course: **Monday, December 14, 2026, 9:00 a.m.** (the 11:00–12:15 TR block). School of Engineering courses are allotted three hours and **do not** offer an alternate exam sitting. Any such change would be announced no later than the withdrawal deadline, Friday October 30.

Term project — 50%

The centerpiece. **Teams of three or four** — see the team-size rule in §12; a team is **required** for students using this course for the CS project requirement. You choose from a menu of eight defined topics, or propose your own with approval.

Deliverable	Share of course grade	When
Proposal — report plus lightning talk	5%	Talks in class **Tue Sep 29**; report and slides both due that day at **11:00 a.m.**
Milestone — presentation plus **running artifact**	10%	*Tentative:* Thu Nov 12
Showcase poster or video file — required of undergraduate teams	(within the final)	**Thu Dec 3 — hard deadline, see §12**
Final — report, presentation, demo, artifacts	35%	*Tentative:* Dec 8 / Dec 10 and finals period

> **Which of these dates can move, and which cannot.** The four presentation dates marked *tentative* are **course-internal and provisional** — they are set by me, and I may shift them by a session or two once I see how team formation and the tool sequence actually go. Any change will be announced in class and on Brightspace with as much notice as I can give, and the **weights will not change** with them. What is **not** tentative is anything set by the department or the registrar: the **December 3 poster deadline**, the **December 10 Showcase**, add/drop, and the withdrawal deadline. Plan against those as fixed.

The proposal talks are deliberately early. They are in class on **Tuesday September 29**, with the report (1–3 pages) and the slides (roughly 5–7) both due at 11:00 a.m. that morning. The point is to make you commit to a team and a topic in September rather than late October: the second most common problem in Fall 2025 was teams starting the formal work too late to recover.

Full instructions, the eight-topic menu, and the grading rubrics are posted on Brightspace and in the course repository.

A change from previous years, stated plainly. Topics are now provided rather than open, and the milestone is a *gate*: by Nov 12 you must have a verifier that runs and produces output. Last year several teams reached November with models trained and nothing verified, and there is not enough semester left after that point to recover. Both changes exist to protect your grade, not to constrain your ambition — every topic on the menu has a genuinely open stretch goal.

Every project must contain both an FM half and an AI half. The most common milestone feedback in Fall 2025 was, verbatim, *"where is the AI?"* The topic menu is constructed so this cannot happen by accident.

Homeworks — 20%

Five homeworks, roughly biweekly, 4% each. One tool per homework: Z3, then nuXmv, then CBMC and abstract interpretation, then Lean 4, then neural network verification. Each has an AI component in which you use an assistant and then *check its work with the tool* — which is the one setting where you can actually find out whether it was right. Full descriptions are posted on Brightspace.

Late policy: four late days for the semester, whole-day units, at most two per homework, no questions asked. After that, 20% per day. Late days do not apply to project deliverables, which are presented in class.

Quizzes — 10%

Five short online quizzes in Brightspace, 10–15 minutes, auto-graded, taken before the class that uses the material. **The lowest score is dropped.** They exist so that lecture can assume you have done the reading, and so you find out early — cheaply — when you have not.

Case studies, discussions, attendance, and participation — 20%

This is a discussion course as much as a lecture course, and the 20% is real.

Structured discussions. Nine sessions during the semester devote roughly 30 minutes to a structured discussion of a genuinely contested question, each followed by a short written reflection of **150–250 words**, due 11:59 p.m. two days after the session — weekly for the first five weeks, then every other week. Each reflection is scored **0–2**, and **the lowest of the nine is dropped**, so eight count. That drop covers one absence with no explanation needed. Format is **think–pair–share**: you think and write alone, argue it out with one partner, then we take it to the room. Topics include: *Has AI ever killed anyone? Are LLMs theft? Are AI's environmental impacts real or overstated? If AI writes all our software, do we still need to learn to code? Can we verify what we cannot specify?* Each has assigned reading — usually one source, under 30 minutes — posted a week ahead on Brightspace. **Come having read it.** The full schedule, readings, and questions are posted on Brightspace and in the course repository.

These are not debate exercises with a house position. Several of them are questions I do not have a settled answer to, and the readings deliberately include sources that cut both ways. What is graded is whether you engage with the strongest version of the view you disagree with.

Case studies. Each student leads or co-leads one in-class case study: an AI or software failure, its mechanism, and what formal method — if any — would have caught it, including the honest answer "none, and here is why."

Everything else. In-class exercises, tool walkthroughs, and peer feedback on project presentations.

Grading scale

A ≥ 93 · A− ≥ 90 · B+ ≥ 87 · B ≥ 83 · B− ≥ 80 · C+ ≥ 77 · C ≥ 73 · C− ≥ 70 · D+ ≥ 67 · D ≥ 63 · D− ≥ 60 · F < 60. The instructor may curve upward; never downward.

Graduate students (CS 5892) — additional required work

CS 3892 and CS 5892 meet together and share one Brightspace site, one schedule, one set of lectures, and the same homeworks and quizzes. **The difference is the term project.** Graduate students carry four additional components, and are held to a higher standard on every rubric row. These are required, not extra credit, and a project without them is incomplete at the graduate level regardless of how good the rest of it is.

Additional requirement	What it means	Where it is graded
Related-work survey	8–12 papers that make an *argument* about where your approach sits — not a list of summaries	Final report — related work
A second, independent evaluation axis	A second dataset, a second tool family, or an ablation that isolates one design choice	Final report — evaluation

Additional requirement	What it means	Where it is graded
Threats to validity	An explicit section naming what could make your conclusion wrong, plus **at least one experiment designed to test it**	Final report — threats
Publication-shaped writing	A credible **8–10 page workshop submission in ACM/IEEE format**, not a class report	Final report — writing

A higher standard throughout. Graduate work is assessed on the same rubrics as undergraduate work, with a higher expectation at every level. A proposal, milestone, or final report that would earn full marks in CS 3892 does not automatically earn them in CS 5892.

Mixed teams are allowed, and normal. In a mixed undergraduate/graduate team the graduate members **own** these components, and that ownership must be stated explicitly in the team's responsibility breakdown. Being on a team where somebody else wrote the survey does not satisfy the requirement.

Full detail — including how each component maps onto the rubric — is in the project topic menu and the project rubrics, both posted on Brightspace and in the course repository.

8. Attendance

Attendance is expected. If you cannot attend, tell me in advance — much of the value of the course is in the in-class activities and discussions, and they cannot be made up asynchronously. Attendance on project presentation days — Sep 29, and tentatively Nov 12, Dec 8 and Dec 10, per the table in §7 — is required; peer feedback on those days is part of the participation grade. If one of those dates moves, the attendance requirement moves with it.

9. Generative AI policy

Generative AI is permitted on all assignments and projects, and its use is expected. This is a course about the trustworthiness of AI-produced artifacts; declining to use the technology would be beside the point. Two conditions:

1. **Disclose it.** Every homework and every project deliverable has an AI-use section. Say what you used and for what. Nobody is penalized for using AI; failing to disclose is an honor-code matter.
2. **You are accountable for what you submit.** "The model produced it" is not a defense. In this course it is a particularly weak one, because for nearly everything we do there is a decision procedure available that will tell you whether the model was right. Run it.

Submitting an unverified AI-generated proof is both an integrity problem and a strategic error: Lean will not accept `sorry`, and the compiler is a more rigorous grader than I am.

One exception: quizzes. Quizzes are **open course materials and open tools — but no chatbots.** Slides, notes, the course repositories, and a running solver are all fair game; an LLM assistant is not. A quiz is a readiness check on your own understanding, and it is the only place in this course where that line is drawn.

Vanderbilt guidance: https://www.vanderbilt.edu/generative-ai/students/ · https://www.vanderbilt.edu/generative-ai/academic-integrity/

10. Honor Code

As a Vanderbilt student it is your responsibility to read and understand the Honor Code: https://studenthandbook.vanderbilt.edu/community-standards

Collaboration and discussion are encouraged throughout. Submitted work must be your own, and external sources — including AI assistants — must be cited.

11. University policies and student support

Students with disabilities. Vanderbilt is committed to equal opportunity for students with disabilities. If you have a physical or learning disability, contact Student Access, which will notify your instructors so that appropriate accommodations can be arranged. Absent notification it is assumed no accommodations are sought. https://www.vanderbilt.edu/student-access/

If you learn best in a particular way, tell me — I would rather adjust than have you struggle, and I can only do that if I know in advance of the assignment. I will keep your needs confidential.

Audio/video recording and intellectual property. Students may not record, reproduce, screenshot, photograph, or distribute any content from their courses, including recordings of in-person or Zoom sessions, except for their own personal educational use. Instructors likewise may not generally share recordings of class activities with people outside the class. Recordings should be deleted at the end of the semester. Failure to follow these policies may be an Honor Code violation or lead to accountability proceedings.

Mental health and wellness. If personal or academic stress is interfering with your ability to work, Vanderbilt's Student Care Network offers a range of support. I am glad to talk about stresses related to this course and can help you connect with the Network. The Office of Student Care Coordination is the first point of contact: https://www.vanderbilt.edu/student-care-network/student-care-coordination/ or 615-343-WELL. Drop-in services: https://www.vanderbilt.edu/student-care-network/drop-in-support/

If you or someone you know needs to speak with a professional counselor immediately, the University Counseling Center offers Crisis Care Counseling. Come to the UCC to be seen by the clinician on call, or phone 615-322-2571. https://www.vanderbilt.edu/student-care-network/university-counseling-center/

Equal Opportunity and Access: https://www.vanderbilt.edu/eoa/
Student Access: https://www.vanderbilt.edu/student-access/
Title IX and student discrimination: https://www.vanderbilt.edu/title-ix/

Mandatory reporting. University faculty, many staff, and some student leaders are required to report incidents of sexual assault, sexual harassment, dating violence, domestic violence, stalking, and child abuse, as well as suspected discrimination on the basis of age, race, color, creed, religion, ancestry, national or ethnic origin, sex or gender, sexual orientation, disability, genetic information, military status, familial status, or other protected categories, to the University's Title IX Coordinator (615-343-9004), as required by University policy and state and federal law.

If you disclose an experience of interpersonal violence or child abuse to classmates with reporting obligations, or to me — in discussion or through a course assignment — your disclosure will be kept as private as possible but may not be able to be kept confidential. This includes incidents on or off campus and incidents that occurred before your time at Vanderbilt. Disclosure of thoughts of harm to yourself or others will also be shared with those responsible for the health and safety of our community.

I am willing to discuss such incidents if you wish, but only in the context of us both understanding my reporting obligations. For confidential conversations, officials in the Student Health Center, the University Counseling Center, and the Office of the Chaplain and Religious Life (when acting as clergy) can maintain confidentiality. The Project Safe Center (crisis line 615-322-7233) has limited confidentiality: staff must report incidents but can do so without identifying the person affected.

12. CS project requirement, Immersion, and the Showcase

This is the authoritative statement of the Immersion and Showcase rules for this course. The project documents in the repository link here rather than restating them, so there is one place to update if anything changes.

The CS Immersion Showcase

Thursday, December 10, 2026, 5:00–7:00 p.m., Adams Atrium, Featheringill Hall.

The undergraduate catalog states, under School of Engineering | Degree Programs in Engineering | Immersion, that *"Senior design projects in the School of Engineering satisfy the final project requirement for Immersion Vanderbilt, with students presenting the results of their design project at the school's Design Day or Computer Science Immersion Showcase."* For Computer Science, a "senior design project" is a CS project course — this one.

Who must present: if you selected **"Senior project design"** as your Immersion experience, you must present your project at the Showcase. If you are satisfying Immersion some other way, or are not using Immersion at all, *presenting at the Showcase* is **not required** — though it is strongly encouraged. The **poster itself is a separate matter and is required of every undergraduate team regardless** (see below).

Graduating seniors: after the event I approve Immersion completion for graduating seniors who selected "Senior project design." The Office of Immersion contacts instructors around the time final grades are due.

Team size — four ideal, three minimum

Four is the intended size. A team may be **three** if any of its members is an undergraduate, and a **graduate-only team of two** is permitted. This rule is here for three reasons:

1. **ABET requires that we assess your ability to function as a member or leader of a team.** Two people is a stretch of that definition.
2. **Attrition.** If one member of a three-person team drops the course, you are a pair. If one member of a *pair* drops, you are alone in November with a project scoped for two.
3. **Showcase logistics.** Fewer, larger teams means each team gets a proper space at the event.

Solo projects require my approval in advance and are held to the same total scope; a graduate pair does not. Students using this course for the CS Project requirement must be on a team.

Posters — file due Thursday, December 3

A poster is required of every undergraduate team in this course, whether or not anyone on the team is using the course for Immersion, and whether or not the team presents at the Showcase. It is part of the final deliverable. Graduate teams are encouraged to produce one and are not required to.

The department pays for poster printing, but only if your file is in on time: it must be submitted by **Thursday, December 3, 2026**, one week before the event. That deadline is not mine and I cannot extend it. A team that misses it can still satisfy the requirement, but pays for its own printing — or submits a short video instead, which is an accepted

substitute.

Read the next section before deciding how much effort to put in: the poster is also the only thing that puts you in front of the award judges.

40AU awards — $3,500 / $1,000 / $500

The top three projects receive monetary awards of **$3,500, $1,000, and $500**, divided equally among team members and deposited into each student's Vanderbilt account. The amounts are set by the donor and can change from year to year.

Only teams that submit a poster file — or another medium that introduces the project, such as a short video — by December 3 will be considered by the judges. There are too many projects for judges to visit them all during a two-hour event, so judging begins beforehand from the submitted material. A team without a poster or video is not in the running, however good the work is.

Judging has historically used the following metrics, weighted differently depending on the mix of projects in a given semester:

Metric	What it asks
Completeness	How complete is the application's functionality
Complexity	How hard is the problem being solved
Ethical implication	How the project may affect people or society, positively or negatively
Interface	How good and user-friendly is the interface
Need identification	Is the need clearly defined — population, problem, impact
Novelty	How new is this relative to what the judge understands as state of the art
Presentation	How well the team presented it
Team size	How many students contributed
Usefulness	How useful would this be in its field

A word of advice specific to this course. Several of those metrics — *interface*, *usefulness*, *completeness* — reward things a verification project does not naturally produce. A judge who is not a formal-methods person will not be moved by a `sorry`-free Lean file or an `unsat` verdict on its own. Build something they can *see*: a visualization of a counterexample trace, a side-by-side of what the verifier caught and what testing missed, a short screen recording of the tool finding a real bug. The technical work is the substance; the demo is what makes the substance legible. Note also that **team size is itself a judging metric**, which is a further argument for four.

Funding — apply early, this is real money

The Office of Immersion funds Immersion-related projects, and **Senior Design projects are eligible for $1,000 per student**. If your project needs cloud credits, compute, hardware, or paid data access, this is the route — and topic 3 (AWS Bedrock) and topic 4 (neural network verification) can both consume real budget.

Mechanics that matter:

- Applications are evaluated on a **monthly** cycle, deadlines at 11:59 p.m., with decisions about three weeks later. Fall 2026 monthly deadlines: **Sep 15 · Oct 15 · Nov 16 · Dec 10**.

- The **fall cycle closes Monday, September 14.** The funding page is inconsistent about this and also shows September 15 — treat **September 14** as the deadline.
- There are **two separate applications** — Immersion Experience funding and Culminating Project funding — and they must be submitted in sequence, never in the same cycle.
- A **funding workshop is required every year**, even if you have attended before.
- Money arrives **four to six weeks after notification**, so September or October, not November.

https://www.vanderbilt.edu/immersion/funding/

Links

- CS Immersion: https://engineering.vanderbilt.edu/academics/undergraduate/cs-immersion/ (short form: http://vu.edu/cs-immersion)
- Immersion Vanderbilt: https://www.vanderbilt.edu/immersion/
- Immersion funding: https://www.vanderbilt.edu/immersion/funding/

Questions about the Showcase or about Immersion generally can come to me, or directly to **Prof. Robert Tairas**, Immersion Coordinator for Computer Science (377 Jacobs Hall).

Syllabus subject to change; changes will be announced in class and on Brightspace.